Costeñol Documentación para estudiar

Aplicación educativa de análisis y ejecución de un lenguaje propio en Python.

Estado documentado al 6 de noviembre de 2026. Esta guía describe los archivos implementados, las reglas adoptadas, su uso y los límites conocidos. No añade funciones al proyecto.

Qué puedes aprender con esta guía

Seguir el recorrido del código desde el editor hasta los resultados; distinguir errores léxicos, sintácticos, semánticos y de ejecución; explicar las funciones principales y preparar una demostración del proyecto.

Qué se ha construido

Un lexer manual, un parser descendente recursivo, un analizador semántico, un intérprete del árbol sintáctico y una interfaz de escritorio con Tkinter. Se validan programas completos antes de comenzar a capturar datos o producir salidas de ejecución.

Precisión para la sustentación: el proyecto se denomina compilador en el curso. Su implementación actual tiene etapas de análisis y un intérprete que recorre el árbol; no genera código máquina, binarios ni un archivo ejecutable del programa Costeñol.

Ruta de lectura

2. Arquitectura y archivos · 3. Reglas del lenguaje · 4. Lexer · 5. Parser · 6. Gramática · 7. Semántica · 8. Alcance e inicialización · 9. Ejecución · 10. Interfaz · 11. Pruebas · 12. Ejemplo guiado · 13. Repaso y límites.

Arquitectura y archivos

```
Código fuente
    |
    v
lexer.py  -> tokens
    |
    v
parser.py -> árbol sintáctico (AST)
    |
    v
semantico.py -> validación de nombres y tipos
    |
    v
interprete.py -> valores, capturas y mensajes
```

La interfaz llama a estas funciones existentes. No contiene otra versión de las reglas del lenguaje. El archivo .cost es texto fuente; su extensión identifica el lenguaje, pero no cambia cómo se procesa.

Archivo	Responsabilidad
lexer.py	Separar caracteres en tokens y ubicar errores léxicos.
parser.py	Comprobar estructura y construir diccionarios que forman el AST.
semantico.py	Comprobar declaraciones, tipos e inicialización.
interprete.py	Evaluar el AST y ejecutar instrucciones.
interfaz.py	Editor, archivos, botones, capturas y resultados.
test_*.py	Pruebas automáticas del parser, semántica, intérprete e interfaz.
ejemplo_*.cost	Programas editables para practicar; pueden contener cambios del estudiante.

Reglas del lenguaje

```
nombre Texto;
edad Entero;
precio Real;
asiste Logico;
edad = Captura.Entero();
precio = 3,5;
nombre = "Ana";
asiste = edad >= 18;
Mensaje.Texto(nombre);
```

La declaración escribe primero el nombre y después el tipo. Declaraciones, asignaciones y mensajes terminan con punto y coma. Si, Sino y FinSi no llevan punto y coma. Las mayúsculas cuentan: Si es reservado; si es un identificador.

Texto, números y comentarios

Los textos se delimitan con comillas dobles rectas. Los reales fuente usan coma decimal: 3,1416. El signo negativo es un operador, no parte del token numérico. Un comentario comienza y termina con // en la misma línea. Dentro de una cadena, // es texto normal.

```
// Comentario completo //
Si (edad >= 18)
    Mensaje.Texto("Mayor de edad");
Sino
    Mensaje.Texto("Menor de edad");
FinSi
```

Reglas concretadas por el equipo

Se adoptaron comparadores ==, !=, <, >, <= y >=; concatenación Texto + Texto; mensajes con expresiones; Sino opcional y bloques anidados. Son decisiones de implementación donde el enunciado no definía todos los detalles. No hay ciclos, funciones de usuario ni operadores lógicos compuestos.

Verdadero y Falso se aceptan como respuestas de Captura.Logico(), pero no son literales reservados del código fuente. Un valor Logico también puede resultar de una comparación.

Análisis léxico: reconocer piezas

Entrada: texto del programa. **Salida:** lista de objetos Token. Un token registra tipo, valor original o lexema, línea y columna; las ubicaciones empiezan en 1.

```
edad = 20;

IDENTIFICADOR  edad   línea 1, columna 1
ASIGNACION     =      línea 1, columna 6
NUM_ENTERO     20     línea 1, columna 8
PUNTO_Y_COMA   ;      línea 1, columna 10
```

Cómo funciona tokenizar(codigo)

1. Divide el código por líneas y recorre cada una con la posición pos. 2. Reconoce comentarios y cadenas antes que operadores. 3. Aplica PATRON en la posición actual para espacios, reales, enteros e identificadores. 4. Consulta PALABRAS para distinguir nombres reservados. 5. Busca símbolos de dos caracteres antes que símbolos simples. 6. Guarda los tokens y avanza; omite espacios y comentarios.

Los reales se reconocen antes que los enteros para conservar 3,5 como una pieza. Los símbolos largos se prueban primero para conservar >= como una comparación y no separarlo en > y =.

Funciones y estructuras que debes ubicar

Token: dataclass inmutable. **PALABRAS:** diccionario de reservadas. **SIMBOLOS:** operadores y signos. **PATRON:** expresión regular con grupos nombrados. **ErrorLexico:** excepción para caracteres desconocidos, cadenas o comentarios sin cierre.

```
python lexer.py ejemplo.cost
```

Ejercicio: prueba edad = @; y después edad = ;. El primer caso falla en el lexer. El segundo se puede tokenizar: falta revisar la estructura en el parser.

Análisis sintáctico y AST

Entrada: tokens. **Salida:** árbol sintáctico abstracto (AST), formado por diccionarios y listas. El AST representa instrucciones y expresiones, no resultados calculados.

El parser descendente recursivo

Cada función reconoce una regla. actual consulta el token vigente; pos indica cuánto se consumió. aceptar avanza si coincide un tipo. exigir utiliza aceptar y genera ErrorSintactico si falta el elemento requerido. Se añade EOF para representar el fin del archivo.

instruccion: reconoce declaración, asignación, mensaje o condicional. **bloque:** reúne instrucciones hasta un cierre permitido. Cada Si procesa sus propios bloques, lo que permite anidamiento.

Prioridad de operaciones

expresion reconoce comparaciones; suma reconoce + y -; producto reconoce * y /; unaria reconoce signos; primaria reconoce literales, variables, capturas y paréntesis. Las multiplicaciones se agrupan antes que las sumas. Las operaciones binarias aritméticas se agrupan por la izquierda.

```
x = 2 + 3 * 4;
```

```
Asignacion: x
  Binaria: +
    Literal: 2
    Binaria: *
      Literal: 3
      Literal: 4
```

El árbol anterior organiza primero 3 * 4. La ejecución posterior calcula 14. Con (2 + 3) * 4 cambia la estructura y el resultado es 20. Los nodos necesarios conservan ubicación para los errores semánticos y de ejecución.

```
python parser.py ejemplo_sintactico.cost --arbol
```

Gramática para estudiar

Esta EBNF resume la estructura implementada. Las llaves significan repetición y los corchetes, una parte opcional; esos signos de la notación no se escriben en los programas.

```
programa = { instruccion } ;
instruccion = declaracion | asignacion
            | mensaje | condicional ;
declaracion = IDENTIFICADOR tipo ";" ;
tipo = "Texto" | "Entero" | "Real" | "Logico" ;
asignacion = IDENTIFICADOR "=" expresion ";" ;
mensaje = "Mensaje" "." "Texto"
          "(" expresion ")" ";" ;
condicional = "Si" "(" expresion ")"
              { instruccion }
              [ "Sino" { instruccion } ] "FinSi" ;
expresion = suma [ comparador suma ] ;
comparador = "==" | "!=" | "<" | ">" | "<=" | ">=" ;
suma = producto { ("+" | "-") producto } ;
producto = unaria { ("*" | "/") unaria } ;
unaria = ("+" | "-") unaria | primaria ;
primaria = NUM_ENTERO | NUM_REAL | CADENA
         | IDENTIFICADOR | captura
         | "(" expresion ")" ;
captura = "Captura" "." tipo "(" ")" ;
```

Qué acepta y qué rechaza

Se admiten bloques vacíos y Sino opcional. Las capturas no reciben argumentos. Solo hay una comparación por nivel: a < b < c no se acepta. Los saltos de línea separan texto, pero no sustituyen al punto y coma.

Ejercicio: localiza en parser.py las funciones suma y producto. Explica por qué suma llama a producto y no directamente a primaria.

Semántica: nombres y tipos

Entrada: AST. **Salida:** tabla de símbolos globales si todo es válido. Esta etapa no calcula valores ni solicita datos. Comprueba el significado estático del programa.

La tabla de símbolos

```
edad Entero;
edad = 20;

"edad": {
    "tipo": "Entero",
    "inicializada": True,
    "linea": 1,
    "columna": 1
}
```

buscar comprueba que exista el nombre. expresion determina el tipo resultante. bloque comprueba declaraciones, asignaciones, mensajes y condiciones. Una declaración no inicializa: leer una variable sin valor previo es error.

Regla	Comportamiento
Asignación	Mismo tipo; también se permite Entero hacia Real. No Real hacia Entero.
+, -, *	Operandos numéricos; resultado Real si participa un Real.
Texto + Texto	Concatena dos textos; no convierte números implícitamente.
División /	Siempre produce Real, incluso 4 / 2.
== y !=	Tipos iguales o pareja Entero/Real. Resultado Logico.
<, >, <=, >=	Solo números. Resultado Logico.
Si	La condición debe tener tipo Logico.
Mensaje.Texto	Acepta expresiones de los cuatro tipos.

Alcance e inicialización segura

Variables visibles y variables locales

Un nombre debe declararse antes de usarse. No se permite declarar otra variable con un nombre ya visible. Las declaraciones nuevas de cada rama de Si/Sino son locales: no existen fuera de esa rama. Las variables exteriores sí se pueden modificar dentro.

```
x Entero;
Si (Captura.Entero() > 0)
    x = 1;
Sino
    x = 2;
FinSi
Mensaje.Texto(x);
```

Este programa es válido: x recibe valor en los dos caminos. En el análisis se crean copias independientes de la tabla para cada rama y, al salir, se combina la inicialización de las variables exteriores.

```
x Entero;
Si (1 == 1)
    x = 1;
FinSi
Mensaje.Texto(x);
```

Este segundo caso se rechaza por lectura sin valor garantizado. El analizador es conservador: no calcula condiciones constantes para descartar caminos. Sin Sino contempla el camino que no entra en el bloque, aunque la comparación escrita parezca siempre verdadera.

Diferencia entre declarar y asignar

x Entero; registra el nombre y tipo. x = 1; asigna un valor. x = x + 1; solo es válido si ya existía un valor para leer el x del lado derecho. Declarar y nunca utilizar una variable sí está permitido.

```
python semantico.py ejemplo_semantico.cost --simbolos
```

Ejecución: interpretar el árbol

Interprete.ejecutar construye el AST y valida todo con AnalizadorSemantico antes de ejecutar una instrucción. Luego bloque recorre las instrucciones y evaluar resuelve expresiones. No se utiliza eval ni exec.

Memoria y orden de evaluación

El entorno guarda tipo y valor de cada variable. Los literales se convierten a valores Python; las operaciones se resuelven con operator. Los operandos se evalúan de izquierda a derecha. Una asignación a Real convierte el valor a float.

En un condicional se ejecuta solo la rama seleccionada. dict(entorno) crea un mapa local, pero comparte las celdas de variables exteriores: sus asignaciones sobreviven; los nombres nuevos quedan locales.

Captura	Formato de respuesta
Texto	Texto incluso vacío; conserva espacios.
Entero	Dígitos con signo opcional: 20 o -5.
Real	Dígitos y coma decimal opcional: 12 o -3,5. No punto, NaN ni infinito.
Logico	Verdadero o Falso, sin distinguir mayúsculas. No 1/0 ni sí/no.

Las entradas numéricas y lógicas eliminan espacios exteriores. Una entrada inválida detiene el programa; no reintenta. Cancelar una captura o llegar al fin de entrada genera ErrorEjecucion.

mostrar convierte los valores a texto: reales con coma y lógicos como Verdadero/Falso. Cada Mensaje.Texto termina con salto de línea. Las cadenas interpretan escapes de salto de línea, tabulación, retorno, comilla y barra invertida; un escape desconocido falla si se evalúa.

```
python interprete.py ejemplo_ejecucion.cost
```

La división entre cero se informa en ejecución. Los reales usan precisión finita de float; se controlan resultados no finitos y algunos desbordamientos. Los mensajes emitidos antes de un error no se revierten.

Interfaz gráfica con Tkinter

```
python interfaz.py
```

Requiere Python 3.10 o posterior con Tcl/Tk. No necesita paquetes externos de pip. python -m tkinter permite comprobar si la instalación puede abrir una ventana.

Acción	Uso y comportamiento
Nuevo / Ctrl+N	Crea un documento. Pregunta por cambios pendientes.
Abrir / Ctrl+O	Lee UTF-8. Filtra .cost y permite Todos los archivos para otros textos.
Guardar / Ctrl+S	Escribe UTF-8; Guardar como permite elegir otra ruta.
Analizar / F5	Comprueba las tres etapas sin ejecutar capturas.
Ejecutar / F6	Valida y ejecuta el texto actual del editor aunque no esté guardado.
Resultados	Muestra mensajes, entradas y errores. Limpiar borra el panel.

Cómo se conecta al motor

Aplicacion.procesar obtiene el código del editor. Para analizar llama a analizar_semantica; para ejecutar llama a ejecutar con las funciones capturar e imprimir. Esa inyección de funciones permite reutilizar el intérprete sin depender de la consola.

capturar usa un diálogo; Cancelar devuelve None, que el intérprete convierte en error. mostrar_error identifica la categoría, extrae línea y columna, resalta la línea y mueve el cursor.

Durante el procesamiento se bloquean el editor y los botones de acciones. Al terminar se restauran. La aplicación pregunta antes de descartar cambios. El procesamiento es síncrono: está pensado para ejercicios cortos, sin depurador ni botón Detener.

Estado de verificación: interfaz implementada; la comprobación visual y las tres pruebas de Tkinter no se completaron en el entorno de desarrollo porque Tcl/Tk no pudo iniciarse. Esto no equivale a una verificación de uso en pantalla.

Errores y pruebas automáticas

Etapa	Ejemplo y causa
Léxica	edad = @; contiene un carácter no reconocido.
Sintáctica	edad = ; carece de expresión a la derecha.
Semántica	edad Entero; edad = "Hola"; asigna Texto a Entero.
Ejecución	Mensaje.Texto(1 / 0); intenta dividir entre cero.

Se informa el primer error. Corrígelo y analiza de nuevo para detectar el siguiente. La ubicación sintáctica puede apuntar al token posterior a una omisión o al fin del archivo; indica dónde se detectó el problema.

```
python -m unittest discover -p "test_*.py" -v
```

Qué se comprueba

test_parser.py: seis pruebas sobre instrucciones, prioridad, anidamiento, comparaciones, errores y ubicación. **test_semantico.py:** diez pruebas sobre nombres, tipos, inicialización, ramas y alcance. **test_interprete.py:** nueve pruebas sobre capturas, operaciones, cadenas, errores, ramas y validación antes de producir efectos.

test_interfaz.py: tres pruebas de integración para análisis y errores, capturas/cancelación y archivos/cambios. Se omiten si no puede iniciarse Tkinter. La última verificación registrada fue 28 pruebas descubiertas: 25 pasaron y 3 gráficas quedaron omitidas. Una omisión no significa que la función haya sido verificada.

Cómo leer una prueba

```
with self.assertRaises(ErrorSintactico):
    analizar("edad = ;")
```

Se espera una excepción. Si el parser acepta ese código, la prueba falla. assertEqual comprueba resultados concretos. FAIL indica una expectativa incumplida; ERROR indica una excepción inesperada; skipped indica que no se ejecutó la prueba.

Recorrido completo de un programa

```
edad Entero;
esMayor Logico;
edad = Captura.Entero();
esMayor = edad >= 18;
Si (esMayor)
    Mensaje.Texto("Mayor de edad");
Sino
    Mensaje.Texto("Menor de edad");
FinSi
```

1. Reconocer

El lexer identifica edad como IDENTIFICADOR, Entero como ENTERO y ; como PUNTO_Y_COMA. Captura.Entero() se separa en CAPTURA, PUNTO, ENTERO, PAREN_ABRE y PAREN_CIERRA.

2. Organizar

El parser crea dos declaraciones, dos asignaciones y un condicional. La comparación edad >= 18 se representa como un nodo Binaria con dos operandos. El bloque del condicional conserva listas distintas para entonces y sino.

3. Validar

La semántica registra edad y esMayor. Captura.Entero() es compatible con edad. La comparación produce Logico y se asigna a esMayor. Ambas variables están inicializadas al ser leídas; la condición es válida.

4. Ejecutar

Se pide la edad. Con entrada 20 se guarda 20, se calcula la comparación como verdadera y se muestra Mayor de edad. Con entrada 15 se ejecuta Sino. Con entrada hola se detiene en la captura con un error de ejecución.

Práctica de estudio

Copia el programa en el editor, pulsa F5 y después F6. Modifica un elemento cada vez: agrega @; borra el valor de una asignación; asigna una cadena a edad; introduce una entrada no numérica. Explica por qué cada fallo pertenece a una etapa diferente.

Repaso para explicar el proyecto

Preguntas y respuestas esenciales

¿Token y lexema son lo mismo? El lexema es el texto original, como edad. El token incluye su categoría y ubicación además del texto.

¿Por qué separar las etapas? Cada una tiene una responsabilidad: reconocer, organizar, validar significado y ejecutar. Permite diagnosticar y probar cada parte.

¿Por qué un AST? Conserva la estructura y prioridad de las operaciones para que semántica e intérprete recorran la misma representación.

¿Tabla de símbolos y entorno son iguales? La tabla semántica describe tipos e inicialización; el entorno de ejecución almacena valores reales del programa.

¿Código válido significa ejecución garantizada? No. Las entradas futuras pueden ser inválidas y una operación puede dividir entre cero.

¿Analizar ejecuta? No. Ejecutar valida primero y después realiza las acciones. Un error semántico en una rama que no se ejecutaría también impide comenzar.

Límites actuales que debes conocer

Solo se atiende el primer error. No hay ciclos, funciones de usuario, arreglos, operadores lógicos compuestos ni literales lógicos reservados. No se genera binario. La interfaz no tiene coloreado de sintaxis, depurador ni botón Detener. No hay recuperación automática de entradas inválidas. La revisión visual de Tkinter sigue pendiente en la instalación local.

Base documental y orden recomendado de lectura

Guía elaborada a partir de lexer.py, parser.py, semantico.py, interprete.py, interfaz.py, los cuatro archivos test_*.py y README.md de la carpeta costenol. Describe el comportamiento del código revisado; algunas reglas son decisiones del equipo, no requisitos textuales del profesor.

Estudia en este orden: Token y tokenizar; aceptar/exigir e instruccion; expresion y prioridad; tabla de símbolos y tipos; unión de ramas; evaluar y bloque; conexión de la interfaz. Después realiza la práctica de la página 12 y explica cada resultado con tus palabras.